Gravitational Wave Data Conditioning

Data conditioning is essential before matched filtering. Raw gravitational wave detector data c
ontains low-frequency noise, instrumental artifacts, and needs proper sampling rates for comput
ational efficiency.

Overview

The conditioning pipeline typically involves:
1. High-pass filtering (remove low-frequency noise below ~15 Hz)
2. Resampling (downsample to appropriate sampling rate)
3. Crop filter wraparound (remove edge artifacts from filtering)
4. PSD estimation (calculate power spectral density for matched filtering)

High-Pass Filtering

Remove low-frequency noise and instrumental artifacts:

```python
from pycbc.filter import highpass

# High-pass filter at 15 Hz (typical for LIGO/Virgo data)
strain_filtered = highpass(strain, 15.0)

# Common cutoff frequencies:
# 15 Hz: Standard for ground-based detectors
# 20 Hz: Higher cutoff, more aggressive noise removal
# 10 Hz: Lower cutoff, preserves more low-frequency content
```

Why 15 Hz? Ground-based detectors like LIGO/Virgo have significant low-frequency noise. Hig
h-pass filtering removes this noise while preserving the gravitational wave signal (typically >
20 Hz for binary mergers).

Resampling

Downsample the data to reduce computational cost:

```python
from pycbc.filter import resample_to_delta_t

# Resample to 2048 Hz (common for matched filtering)
delta_t = 1.0 / 2048
strain_resampled = resample_to_delta_t(strain_filtered, delta_t)

# Or to 4096 Hz for higher resolution
delta_t = 1.0 / 4096
strain_resampled = resample_to_delta_t(strain_filtered, delta_t)

# Common sampling rates:
# 2048 Hz: Standard, computationally efficient
# 4096 Hz: Higher resolution, better for high-mass systems
```

Note: Resampling should happen AFTER high-pass filtering to avoid aliasing. The Nyquist fre
quency (half the sampling rate) must be above the signal frequency of interest.

Crop Filter Wraparound

Remove edge artifacts introduced by filtering:

```python
# Crop 2 seconds from both ends to remove filter wraparound
conditioned = strain_resampled.crop(2, 2)

# The crop() method removes time from start and end:
# crop(start_seconds, end_seconds)
# Common values: 2-4 seconds on each end

# Verify the duration
print(f"Original duration: {strain_resampled.duration} s")
print(f"Cropped duration: {conditioned.duration} s")
```

Why crop? Digital filters introduce artifacts at the edges of the time series. These artifa
cts can cause false triggers in matched filtering.

Power Spectral Density (PSD) Estimation

Calculate the PSD needed for matched filtering:

```python
from pycbc.psd import interpolate, inverse_spectrum_truncation

# Estimate PSD using Welch's method
# seg_len: segment length in seconds (typically 4 seconds)
psd = conditioned.psd(4)

# Interpolate PSD to match data frequency resolution
psd = interpolate(psd, conditioned.delta_f)

# Inverse spectrum truncation for numerical stability
# This limits the effective filter length
psd = inverse_spectrum_truncation(
    psd,
    int(4 * conditioned.sample_rate),
    low_frequency_cutoff=15
)

# Check PSD properties
print(f"PSD length: {len(psd)}")
print(f"PSD delta_f: {psd.delta_f}")
print(f"PSD frequency range: {psd.sample_frequencies[0]:.2f} - {psd.sample_frequencies[-1]:.2f}
 Hz")
```

PSD Parameters Explained

- **Segment length (4 seconds)**: Longer segments give better frequency resolution but fewer av
erages. 4 seconds is a good balance.
- **Low frequency cutoff (15 Hz)**: Should match your high-pass filter cutoff. Frequencies belo
w this are not well-characterized.

Best Practices

1. **Always high-pass filter first**: Remove low-frequency noise before resampling
2. **Choose appropriate sampling rate**: 2048 Hz is standard, 4096 Hz for high-mass systems
3. **Crop enough time**: 2 seconds is minimum, but may need more for longer templates
4. **Match PSD cutoff to filter**: PSD low-frequency cutoff should match high-pass filter frequ
ency
5. **Verify data quality**: Plot the conditioned strain to check for issues

Dependencies

```bash
pip install pycbc
```

References

- [PyCBC Tutorial: Waveform & Matched Filter](https://colab.research.google.com/github/gwastro/
pycbc-tutorials/blob/master/tutorial/3_WaveformMatchedFilter.ipynb) - Practical tutorial coveri
ng data conditioning, PSD estimation, and matched filtering
- [PyCBC Filter Documentation](https://pycbc.org/pycbc/latest/html/pycbc.filter.html)
- [PyCBC PSD Documentation](https://pycbc.org/pycbc/latest/html/pycbc.psd.html)

Common Issues

Problem: PSD estimation fails with "must contain at least one sample" error
- **Solution**: Ensure data is long enough after cropping (need several segments for Welch meth
od)

Problem: Filter wraparound artifacts in matched filtering
- **Solution**: Increase crop amount or check that filtering happened before cropping

Problem: Poor SNR due to low-frequency noise
- **Solution**: Increase high-pass filter cutoff frequency or check PSD inverse spectrum trunca
tion